Práctica evaluable 2 – Ping Pong en ensamblador RISC-V sobre ESP32-C6

1. Objetivo general

Completar en ensamblador RISC-V un juego tipo Pong para dos jugadores que se ejecuta sobre una placa ESP32-C6, con una pantalla OLED de 0,96" (SSD1306, 128×64 píxeles) y cuatro pulsadores montados sobre una protoboard.

En la Práctica 1 el programa corría en el simulador RARS. Ahora corre en hardware real: se mantiene la forma de pedir servicios al sistema (ecall pasa a ser call os_ecall) y los pulsadores se leen como entonces el teclado: accediendo a una dirección de memoria (MMIO). El alumno realiza dos tareas:

1. **Montar el circuito** en la protoboard (pantalla y pulsadores).
2. **Programar la lógica del juego** en el fichero miprograma.S: posición de las palas, movimiento de la pelota, rebotes, marcador y final de partida.

2. Material necesario

Cantidad	Elemento	Observaciones
1	Placa ESP32-C6 (p. ej. ESP32-C6-DevKitC-1)	Programación por USB
1	Cable USB de datos	No vale un cable solo de carga
1	Pantalla OLED 0,96" SSD1306, 128×64, I2C	Dirección I2C 0x3C, 4 pines (GND, VCC, SCL, SDA)
4	Pulsadores para protoboard	Dos por jugador: arriba y abajo
1	Protoboard	
10 o más	Cables Dupont	Macho-macho
1	Ordenador con VS Code y ESP-IDF	Versión 5.2 o superior; instalación según la *Guía de instalación VS Code + ESP-IDF para ESP32-C6 (lab_2)*

No hacen falta resistencias externas: el programa activa las resistencias de pull-up internas de los pines de los pulsadores.

3. Montaje del circuito

Monta el circuito sobre la protoboard con la placa **desconectada del USB**, y conecta la placa solo cuando hayas revisado todas las conexiones.

Elemento	Pin del módulo	Pin de la ESP32-C6	Observaciones
OLED SSD1306	VCC	3V3	No usar 5 V salvo que el módulo lo admita
OLED SSD1306	GND	GND	
OLED SSD1306	SDA	GPIO6	Bus I2C, dirección 0x3C
OLED SSD1306	SCL	GPIO7	Bus I2C

Pulsador jugador 1, arriba	un terminal	GPIO2	El otro terminal a GND
Pulsador jugador 1, abajo	un terminal	GPIO3	El otro terminal a GND
Pulsador jugador 2, arriba	un terminal	GPIO18	El otro terminal a GND
Pulsador jugador 2, abajo	un terminal	GPIO19	El otro terminal a GND

Puntos a tener en cuenta:

- Cada pulsador va **entre el pin GPIO y GND**. Con la resistencia de pull-up interna activada, un pulsador sin pulsar se lee como 1 y pulsado como 0.
- Une los cuatro terminales de GND a una misma línea de la protoboard y conecta esa línea a un pin GND de la placa.
- Los pines indicados son los que usan main.c y miprograma.S. Si cambias alguno, hay que cambiarlo a la vez en los dos ficheros.
- Consulta el fichero pinout.pdf del repositorio para localizar los pines en la placa. Los pines del módulo OLED suelen ir en el orden GND, VCC, SCL, SDA, distinto del de la tabla: fíjate en la serigrafía de tu módulo. En muchos módulos de 0,96" las dos primeras filas de píxeles son amarillas y las seis siguientes azules.

Comprobación previa. Antes de escribir código, compila y carga el proyecto sin modificar. Si el montaje es correcto, el monitor serie no muestra mensajes de error del tipo oled_cmd. El esqueleto todavía no dibuja ni borra nada, así que la pantalla puede mostrar puntos aleatorios: es normal.

4. Descripción del comportamiento requerido

El juego es un Pong para dos jugadores. La pantalla se trata como una cuadrícula de **16 columnas × 8 filas** de *tiles* de 8×8 píxeles (columnas 0 a 15, filas 0 a 7). Todo lo que se dibuja es un tile.

Estado inicial. Al arrancar, el programa:

- Borra la pantalla.
- Dibuja la red en el centro (por ejemplo, en la columna 7, filas 0 a 7).
- Dibuja la pala izquierda en la columna 0 y la pala derecha en la columna 15, ambas a mitad de altura.
- Dibuja la pelota cerca del centro.
- Muestra el marcador 0 – 0 con los tiles de dígitos (por ejemplo, en la fila 0, columnas 5 y 10).

Palas. Cada jugador mueve su pala con dos pulsadores. Mientras un pulsador esté pulsado, la pala se desplaza una fila por cada ciclo del bucle principal (100 ms). La pala no puede salir de las filas 0 a 7. Al moverse, se borra el tile de la posición anterior y se dibuja en la nueva.

Jugador	Pala	Arriba	Abajo
1	Izquierda (columna 0)	GPIO2	GPIO3

2	Derecha (columna 15)	GPIO18 GPIO19

Pelota. Se mueve una casilla por ciclo, según su desplazamiento horizontal y vertical (a5 y a6, con valores −1 o +1).

- Rebota en las filas 0 y 7 (se invierte el desplazamiento vertical).
- Cuando llega a la columna de una pala, si la pala está en la misma fila que la pelota, rebota (se invierte el desplazamiento horizontal).
- Si la pala no está en esa fila, el jugador contrario consigue un punto. Se actualiza el marcador y la pelota vuelve al centro para un nuevo saque (la dirección de saque la decide el alumno).
- Al pasar por la columna de la red, la red debe volver a dibujarse cuando la pelota la abandone.

Fin de partida. Gana el primer jugador que llegue a **5 puntos** (valor definido con una constante .equ, para poder cambiarlo; el máximo representable es 9, porque solo hay tiles para los dígitos 0 a 9). Al terminar:

- Se muestra el ganador con los tiles 15 y 16 (signos de admiración de apertura y cierre), junto a su marcador.
- El programa finaliza con el servicio SYS_EXIT. La pantalla conserva la imagen.

5. Código entregado y servicios del sistema

El proyecto lab_2 se entrega casi terminado: solo se trabaja en miprograma.S. Estructura de la carpeta:

Fichero	Contenido	¿Se modifica?
CMakeLists.txt y main/CMakeLists.txt	Configuración del proyecto ESP-IDF	No
main/main.c	Arranque del hardware (I2C, pantalla SSD1306, pines de los pulsadores) y servicios del sistema (os_syscall)	No
main/miprograma.S	Tabla de tiles, bucle principal, lectura de pulsadores, retardo y rutinas del juego	Sí, solo los bloques marcados *CÓDIGO A COMPLETAR POR EL ALUMNO*

Bucle principal. main.c inicializa el hardware y salta a game_run, que no retorna. game_run activa el contador de ciclos, llama una vez a inicializa_juego y después repite, cada 100 ms: procesa_teclas, delay_ms y mueve_pelota.

Lectura de los pulsadores. procesa_teclas lee directamente el registro de entrada GPIO, en la dirección 0x6009103C, y lo invierte para que un 1 signifique "pulsado". El bit *n* corresponde al GPIO *n*. Según el bit activo, salta a izq_up (GPIO2, máscara 0x4), izq_down (GPIO3, 0x8), der_up (GPIO18, 0x40000) o der_down (GPIO19, 0x80000). El esqueleto atiende una sola pulsación por ciclo.

Servicios del sistema. Se piden como ecall en RARS: número de servicio en a7, argumentos en a0–a2 y call os_ecall. El resultado vuelve en a0.

a7	Servicio	Argumentos	Definido en miprograma.S
100	Dibujar un tile	a0 = columna (0 a 15), a1 = fila (0 a 7), a2 = dirección de los 8 bytes del tile	SYS_OLED_TILE
101	Borrar la pantalla	—	SYS_OLED_CLEAR
10	Terminar el programa	—	SYS_EXIT
1 / 4 / 11 / 34	Imprimir entero / cadena / carácter / hexadecimal en el monitor serie	a0	No: añade tu .equ
102	Volcar un buffer de 1024 bytes a la pantalla	a0 = dirección del buffer	No: añade tu .equ

Tiles disponibles. Cada tile ocupa 8 bytes y el tile *N* empieza en la dirección tiles + N*8. Se pueden definir hasta 256 tiles.

Nº	Tile	Nº	Tile
0 a 9	Dígitos 0 a 9	13	Red
10	Pelota	14	Tile vacío (para borrar)
11	Pala izquierda	15	Abre admiración (ganador)
12	Pala derecha	16	Cierra admiración (ganador)

Ejemplo. Dibujar el tile 2 en la columna 5, fila 3:

```
li   a0, 5          # columna
li   a1, 3          # fila
la   a2, tiles      # base de la tabla de tiles
addi a2, a2, 16     # tile N empieza en tiles + N*8 (aqui N = 2)
li   a7, SYS_OLED_TILE  # 100
call os_ecall
```

Reglas de uso de registros.

- os_ecall conserva todos los registros salvo a0.
- a3 y a4 guardan la columna y la fila de la pelota; a5 y a6, su desplazamiento. No los uses para otra cosa sin guardarlos.
- Toda subrutina que llame a otra con call o jal debe guardar ra en la pila y restaurarlo antes de ret, como hacen los esqueletos.
- Si usas registros s0 a s11, guárdalos y restáuralos (convenio ABI de RISC-V).

6. Requisitos y tareas

6.1. Rutinas a completar. Solo se escribe código en los bloques marcados *CÓDIGO A COMPLETAR POR EL ALUMNO* de estas seis rutinas (las variables y subrutinas auxiliares que añadas se describen en 6.2 y 6.3):

Rutina	Qué debe hacer
inicializa_juego	Borrar la pantalla; dibujar la red, las dos palas y la pelota; poner a 0 los marcadores (puntos_izquierda y puntos_derecha) y dibujarlos; dar valores iniciales a a3, a4 (posición de la pelota) y a5, a6 (su desplazamiento)
izq_up / izq_down	Mover la pala izquierda una fila arriba o abajo, sin salirse de las filas 0 a 7: borrar el tile anterior y dibujar el nuevo
der_up / der_down	Lo mismo para la pala derecha
mueve_pelota	Borrar la pelota, calcular su nueva posición, resolver rebotes y puntos, redibujar la red y la pelota, actualizar el marcador y terminar la partida cuando un jugador llegue a 5 puntos

6.2. Datos. La posición de cada pala no está definida en el esqueleto: debes reservar tu propia variable en la sección .data (o usar registros s, siguiendo el convenio ABI). Los marcadores ya existen como puntos_izquierda y puntos_derecha. Los valores fijos (columnas de las palas, filas máximas, puntos para ganar) se definen con .equ, no como números sueltos en el código.

6.3. Estructura modular. Se pueden añadir las subrutinas auxiliares que se necesiten. Se recomienda, como mínimo, una subrutina común para dibujar un tile a partir de su número, columna y fila, que evite repetir el bloque de llamada a os_ecall.

6.4. Gestión de registros. Debe respetarse el convenio ABI de RISC-V:

- Guardar y restaurar ra y los registros s que se usen.
- Pasar los argumentos en a0, a1, etc., y devolver los valores en a0.
- No alterar a3 a a6 fuera de inicializa_juego y mueve_pelota.

6.5. Comentarios. El código debe incluir comentarios que expliquen:

- El propósito de cada subrutina.
- El uso de los registros.
- Las reglas de rebote y de puntuación.

6.6. Restricciones.

- No se modifica main.c, ni los pines, ni la frecuencia de la CPU (160 MHz; delay_ms depende de ella).
- El bucle principal conserva el retardo de 100 ms entre movimientos.

7. Orden de trabajo recomendado:

1. Monta el circuito y comprueba que el proyecto sin modificar compila, se carga y no da errores de pantalla.
2. Implementa inicializa_juego y comprueba que aparecen la red, las palas, la pelota y el marcador.
3. Implementa las cuatro rutinas de las palas y comprueba los límites superior e inferior.

4. Implementa mueve_pelota primero con rebotes en las filas 0 y 7, después con rebotes en las palas.
5. Añade el marcador y el final de partida.

8. Material entregable

1. **Fichero miprograma.S** con el código ensamblador completo. Debe compilar sin errores en el proyecto lab_2 sin tocar el resto de ficheros.
2. **Evidencia del montaje y del funcionamiento:** Se mostrará funcionando en clase.
3. **Informe breve** (máximo una página).

9. Información adicional



